42

崩溃一致性：FSCK 和日志

我们已经看到，文件系统（file system）管理着一组数据结构（data structure），用来实现人们所期望的那些抽象（abstraction）：文件、目录（directory），以及支撑文件系统基本抽象所需的其他所有元数据（metadata）。文件系统的数据结构必须**持久**（persist），也就是要长期保存下来，存放在断电（power loss）后也不会丢失数据的设备上，比如硬盘或基于闪存（flash）的 SSD；这一点和大多数数据结构不同，比如运行中的程序（running program）放在内存里的那些。

文件系统面临的一大挑战是：在可能发生**断电**（power loss）或**系统崩溃**（system crash）的情况下，如何更新持久的数据结构。具体来说，如果磁盘上的结构正更新到一半，有人被电源线绊了一下，机器断电了，会怎样？或者操作系统遇到 bug 崩溃了，又会怎样？因为随时可能断电或崩溃，更新持久数据结构相当棘手，这就引出了文件系统实现中一个有趣的新问题，叫作**崩溃一致性问题**（crash-consistency problem）。

这个问题很容易理解。假设为了完成某个操作，你必须更新磁盘上的两个结构 A 和 B。由于磁盘一次只处理一个请求，两个请求中总有一个先到达磁盘（A 或 B）。如果系统在其中一个写入完成后崩溃或断电，磁盘上的结构就会处于**不一致**（inconsistent）的状态。于是，我们遇到了一个所有文件系统都必须解决的问题：

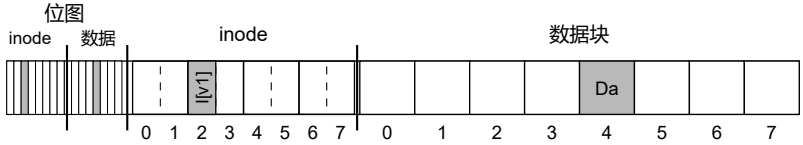
> 关键问题：如何在可能崩溃的情况下更新磁盘
> 系统可能在任意两次写入之间崩溃或断电，因此磁盘上的状态可能只更新了一部分。崩溃之后，系统重新启动，并希望再次挂载（mount）文件系统（以便访问文件等）。既然崩溃可能发生在任意时刻，我们如何确保文件系统在磁盘上的映像始终处于合理的状态？

在本章中，我们会更详细地描述这个问题，并看看文件系统用过哪些方法来解决它。我们先研究较老的文件系统采用的方法，叫作 **fsck**，即**文件系统检查程序**（file system checker）。然后我们转向另一种方法，叫作**日志**（journaling），也叫**预写日志**（write-ahead logging）。这种技术会给每次写入增加一点开销，但能更快地从崩溃或断电中恢复。我们会讨论日志的基本机制，包括比较现代的日志文件系统（journaling file system）Linux ext3 [T98, PAA05] 所实现的几种不同的日志方式。

42.1 一个详细的例子

为了开始研究日志，我们先来看一个例子。我们需要一个会更新磁盘上某些结构的**工作负载**（workload）。这里假设工作负载很简单：往一个已有的文件末尾追加一个数据块（data block）。追加的过程是：打开文件，调用 `lseek()` 把文件偏移量（file offset）移到文件末尾，然后向文件发出一次 4 KB 的写入，最后关闭文件。

我们再假设磁盘上使用的是标准的简单文件系统结构，和我们之前见过的文件系统类似。这个小例子包括一个 **inode 位图**（inode bitmap，只有 8 位，每个 inode 一位）、一个**数据位图**（data bitmap，也是 8 位，每个数据块一位）、inode（共 8 个，编号 0 到 7，分布在四个块中），以及数据块（共 8 个，编号 0 到 7）。下面是这个文件系统的示意图：



从图中可以看出，系统分配了一个 inode，它的 inode 号（inode number）是 2，并在 inode 位图中做了标记；还分配了一个数据块（数据块 4），同样在数据位图（data bitmap）中做了标记。这个 inode 记作 I[v1]，因为它是这个 inode 的第一个版本；上面那个工作负载很快就会更新它。

我们也来看看这个简化的 inode 内部。I[v1] 里面是这样的：

```
owner       : remzi
permissions : read-write
size        : 1
pointer     : 4
pointer     : null
pointer     : null
pointer     : null
```

在这个简化的 inode 中，文件的 `size` 是 1（分配了一个块），第一个直接指针（direct pointer）指向块 `4`（文件的第一个数据块 Da），另外三个直接指针都设为 `null`（表示没有使用）。当然，真实的 inode 有多得多的字段；更多信息请参阅前面几章。

往文件里追加数据，就是给它加一个新的数据块，因此必须更新磁盘上的三个结构：inode（它必须指向新块，并且因为追加而记录新的、更大的文件大小）、新的数据块 Db，以及新版本的数据位图（记作 B[v2]），用来表示新数据块已经分配出去了。

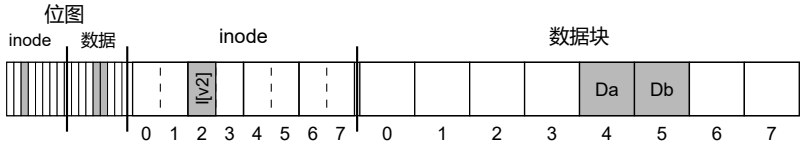
所以，系统内存里有三个块（block）要写到磁盘上。更新后的 inode（inode 版本 2，简称 I[v2]）现在是这样的：

```
owner       : remzi
permissions : read-write
size        : 2
pointer     : 4
pointer     : 5
```

```
pointer      : null
pointer      : null
```

更新后的数据位图（data bitmap）B[v2] 现在是这样的：00001100。最后是数据块（Db），里面装的就是用户放进文件里的任何东西。也许是偷来的音乐?

我们希望文件系统最终在磁盘上的映像是这样的：



为了完成这一转变，文件系统必须对磁盘执行三次独立的写入，inode（I[v2]）、位图（B[v2]）和数据块（Db）各一次。注意，用户发出 `write()` 系统调用（system call）时，这些写入通常不会立即发生；脏的 inode、位图（bitmap）和新数据会先在内存（**页缓存**，page cache，或**缓冲区缓存**，buffer cache）中待上一段时间；然后，等文件系统最终决定把它们写到磁盘时（比如 5 秒或 30 秒之后），才会向磁盘发出所需的写请求。遗憾的是，崩溃可能会发生，打断这些磁盘更新。具体来说，如果这三次写入只完成了一两次就发生崩溃，文件系统就可能处于一种奇怪的状态。

崩溃场景

为了更好地理解这个问题，我们来看几个崩溃场景的例子。假设只有一次写入成功了；那么有三种可能的结果，列举如下：

- **只有数据块（Db）写到了磁盘上**。在这种情况下，数据在磁盘上，但没有 inode 指向它，也没有位图表示这个块已被分配。因此，这就好像那次写入从未发生过一样。从文件系统崩溃一致性（crash consistency）的角度看，这种情况根本不是问题[1]。
- **只有更新后的 inode（I[v2]）写到了磁盘上**。在这种情况下，inode 指向了 Db 本该写入的磁盘地址（5），但 Db 还没有写到那里。因此，如果我们相信这个指针，就会从磁盘上读到**垃圾**（garbage）数据，也就是磁盘地址（disk address）5 原来的内容。此外，我们还遇到了一个新问题，叫作**文件系统不一致**（file-system inconsistency）。磁盘上的位图告诉我们数据块 5 还没有被分配，inode 却说它已经分配了。位图和 inode 之间的这种矛盾，就是文件系统数据结构中的不一致；要使用这个文件系统，我们必须设法解决这个问题（下文会详细讨论）。
- **只有更新后的位图（B[v2]）写到了磁盘上**。在这种情况下，位图表示块 5 已被分配，但没有 inode 指向它。因此，文件系统又一次不一致了；如果不加处理，这次写入会造成 **space leak**（空间泄漏），因为文件系统永远不会再使用块 5。

在这次向磁盘写入三个块的尝试中，还有另外三种崩溃场景。在这些情况下，两次写入成功了，最后一次失败了：

[1]不过，这对用户来说可能是个问题：用户刚刚丢了一些数据！

- **inode（I[v2]）和位图（B[v2]）写到了磁盘上，但数据（Db）没有**。在这种情况下，文件系统的元数据是完全一致的：inode 有一个指向块 5 的指针，位图表示块 5 正在使用，所以从文件系统元数据的角度看，一切正常。但有一个问题：块 5 里又是垃圾。
- **inode（I[v2]）和数据块（Db）写到了磁盘上，但位图（B[v2]）没有**。在这种情况下，inode 指向了磁盘上正确的数据，但 inode 和旧版本的位图（B1）之间又出现了不一致。因此，我们还是得先解决这个问题，才能使用文件系统。
- **位图（B[v2]）和数据块（Db）写到了磁盘上，但 inode（I[v2]）没有**。在这种情况下，inode 和数据位图（data bitmap）之间又出现了不一致。不过，虽然这个块已经写入、位图也表明它在使用中，我们却不知道它属于哪个文件，因为没有任何 inode 指向这个文件。

崩溃一致性问题

从这些崩溃场景中，希望你能看出，崩溃会给磁盘上的文件系统映像带来许多问题：文件系统数据结构可能不一致；可能出现 space leak；可能把垃圾数据返回给用户；等等。理想情况下，我们希望**原子地**（atomically）把文件系统从一个一致的状态（例如，文件追加之前）转移到另一个一致的状态（例如，inode、位图和新数据块都写到磁盘之后）。遗憾的是，这一点不容易做到，因为磁盘一次只提交一个写入，而这些更新之间可能发生崩溃或断电。我们把这个一般性的问题称为**崩溃一致性问题**（也可以叫它 **consistent-update problem**）。

42.2 方案一：文件系统检查程序

早期的文件系统对崩溃一致性（crash consistency）采取了一种简单的方法。基本上，它们决定先让不一致发生，以后（重启时）再修复。这种“偷懒”做法的一个经典例子，就是做这件事的工具：**fsck**[2]。fsck 是一个 UNIX 工具，用来找出这类不一致并加以修复 [MK96]；不同的系统上也有类似的工具，用来检查和修复磁盘分区。注意，这种方法并不能解决所有问题；比如上面那种情况：文件系统看起来是一致的，但 inode 指向了垃圾数据。它真正的目标只是确保文件系统的元数据在内部是一致的。

正如 McKusick 和 Kowalski 的论文 [MK96] 所总结的，fsck 工具分几个阶段运行。它在文件系统挂载、可供使用之前运行（fsck 假定它运行时没有其他文件系统活动在进行）；运行结束后，磁盘上的文件系统应该是一致的，因此可以开放给用户访问。

下面是 fsck 所做工作的基本概述：

- **超级块：** fsck 首先检查超级块（superblock）看起来是否合理，主要是做一些健全性检查，比如确保文件系统的大小大于已分配的块数。这些健全性检查的目的通常是找出可疑的（损坏的）超级块；在这种情况下，系统（或管理员）可能会决定改用超级块的某个备份副本。
- **空闲块：** 接下来，fsck 扫描 inode、间接块（indirect block）、二级间接块（double indirect block）等，从而弄清楚文件系统中当前有哪些块已被分配。它利用这些信息

[2] 读作“eff-ess-see-kay”或“eff-ess-check”；如果你不喜欢这个工具，也可以读作“eff-suck”（suck 意为“烂透了”）。没错，严肃的专业人士也用这个说法。

生成一份正确的分配位图；因此，如果位图和 inode 之间有任何不一致，都以 inode 中的信息为准来解决。对所有 inode 也做同样的检查，确保所有看起来正在使用的 inode 都在 inode 位图中做了相应标记。

- **inode 状态：** 检查每个 inode 有没有损坏或其他问题。例如，fsck 会确保每个已分配的 inode 都有一个有效的类型字段（例如普通文件、目录、符号链接（symbolic link）等）。如果 inode 的字段有不易修复的问题，fsck 就会把这个 inode 视为可疑的并清除它；inode 位图也会相应更新。
- **inode 链接：** fsck 还会核对每个已分配 inode 的链接计数（link count）。你可能还记得，链接计数表示有多少个不同的目录包含对这个文件的引用（即链接）。为了核对链接计数，fsck 从根目录（root directory）开始扫描整个目录树（directory tree），为文件系统中的每个文件和目录自己算出一份链接计数。如果新算出的计数和 inode 中记录的不一致，就必须采取纠正措施，通常是修正 inode 中的计数。如果发现了一个已分配的 inode，却没有任何目录引用它，就把它移到 `lost+found` 目录中。
- **重复：** fsck 还会检查重复的指针，即两个不同的 inode 指向同一个块的情况。如果其中一个 inode 明显是坏的，就可以把它清除。另外，也可以把被指向的块复制一份，按需让每个 inode 都有自己的副本。
- **坏块：** 扫描所有指针的列表时，还会检查坏块指针（bad block pointer）。如果一个指针明显指向了有效范围之外的东西，例如它的地址所指的块超出了分区的范围，这个指针就被认为是“坏”的。在这种情况下，fsck 做不了什么太聪明的事；它只是把这个指针从 inode 或间接块（indirect block）中删掉（清除）。
- **目录检查：** fsck 并不理解用户文件的内容；但目录里存放的是由文件系统自己创建的、格式特定的信息。因此，fsck 会对每个目录的内容做额外的完整性（integrity）检查，确保“.”和“..”是最前面的两个目录项（directory entry），确保目录项中引用的每个 inode 都已分配，并确保在整个层次结构中，任何目录都只被链接一次。

可以看出，要构建一个能用的 fsck，需要对文件系统有非常细致的了解；确保这样一段代码在所有情况下都能正确工作，可能相当有挑战性 [G+08]。然而，fsck（以及类似的方法）还有一个更大、或许也更根本的问题：它们太慢了。如果磁盘卷（disk volume）非常大，扫描整个磁盘来找出所有已分配的块、读取整个目录树，可能要花好几分钟甚至几个小时。随着磁盘容量增长、RAID 越来越普及，fsck 的性能变得让人无法接受（尽管最近有一些进展 [M+13]）。

从更高的层面看，fsck 的基本前提似乎有点不合情理。想想上面的例子，只有三个块要写到磁盘上；为了修复仅仅更新三个块时出现的问题，却要扫描整个磁盘，代价高得离谱。这种情况就好比你把钥匙掉在了卧室的地板上，然后启动一个把整栋房子翻个遍找钥匙的恢复算法，从地下室开始，一个房间一个房间地找过去。这样做能找到，但太浪费了。因此，随着磁盘（和 RAID）越来越大，研究人员和从业者开始寻找其他解决方案。

 © 2008–23, ARPACI-DUSSEAU

42.3 方案二：日志（或预写日志）

对于 consistent-update problem，最流行的解法大概借鉴自数据库管理系统（database management system）领域的一个思想。这个思想叫作**预写日志**（write-ahead logging），正是为了解决这类问题而发明的。在文件系统中，由于历史原因，我们通常把预写日志（write-ahead logging）叫作**日志**（journaling）。第一个这样做的文件系统是 Cedar [H87]，不过如今许多现代文件系统都用了这个思想，包括 Linux ext3 和 ext4、reiserfs、IBM 的 JFS、SGI 的 XFS 以及 Windows NTFS。

基本思想如下。更新磁盘时，在原地覆盖那些结构之前，先写下一条小小的记录（写在磁盘上的另一个地方，一个大家都知道的位置），描述你接下来要做什么。写这条记录就是“预写”（write ahead）的部分，而我们把它写到一个组织成“日志”（log）的结构里；于是就有了“预写日志”（write-ahead logging）这个名字。

把这条记录写到磁盘上，就能保证：如果在更新（覆盖）结构的过程中发生了崩溃，你可以回过头去看看你写下的记录，然后再试一次；这样一来，崩溃之后你就能确切地知道要修复什么（以及怎么修复），而不必扫描整个磁盘。因此，日志在设计上就是在更新时多做一点工作，来大大减少恢复时需要做的工作。

下面我们来描述 **Linux ext3** 这个流行的日志文件系统（journaling file system）是如何把日志纳入文件系统的。它在磁盘上的大部分结构都和 **Linux ext2** 相同，例如，磁盘被划分为若干块组，每个块组（block group）包含一个 inode 位图、一个数据位图（data bitmap）、若干 inode 和数据块。新增的关键结构就是日志本身，它在分区内或另一个设备上占据一小块空间。因此，一个（没有日志的）ext2 文件系统看起来是这样的：



假设日志放在同一个文件系统映像中（不过有时它也放在单独的设备上，或者作为文件系统中的一个文件），带日志的 ext3 文件系统看起来是这样的：

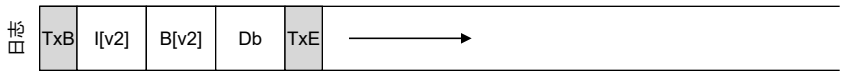


真正的区别只是多了一个日志，当然，还有它的用法。

数据日志

我们来看一个简单的例子，以理解**数据日志**（data journaling）是怎样工作的。数据日志（data journaling）是 Linux ext3 文件系统提供的一种模式，下面的讨论大多以它为基础。

假设我们又要做那个典型的更新：把 inode（I[v2]）、位图（B[v2]）和数据块（Db）写到磁盘上。现在，在把它们写到磁盘上的最终位置之前，我们要先把它们写到日志（log，也叫 journal）里。在日志中，这些块看起来是这样的：



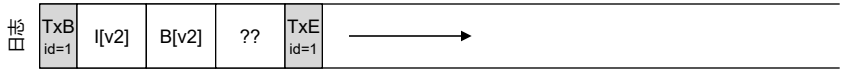
可以看到，这里我们写了五个块。事务开始块（transaction begin block，TxB）告诉我们这次更新的情况，包括即将对文件系统做哪些更新（例如，块 I[v2]、B[v2] 和 Db 的最终地址），以及某种**事务标识符**（transaction identifier，**TID**）。中间三个块就是这些块本身的确切内容；这叫作**物理日志**（physical logging），因为我们把更新的确切物理内容放进了日志。另一种思路是**逻辑日志**（logical logging），它把更新的一种更紧凑的逻辑表示放进日志，例如“这次更新要把数据块 Db 追加到文件 X”，这种方式复杂一些，但能节省日志空间，或许还能提高性能。最后一个块（TxE）标志着这个事务（transaction）的结束，其中也包含 TID。

一旦这个事务安全地写到了磁盘上，我们就可以去覆盖文件系统中的旧结构了；这个过程叫作**检查点**（checkpointing）。因此，要对文件系统做**检查点**（checkpoint），也就是让它和日志中待完成的更新保持一致，我们就把 I[v2]、B[v2] 和 Db 写到上面看到的它们在磁盘上的位置；如果这些写入成功完成，我们就成功地对文件系统做了检查点（checkpoint），基本上就大功告成了。于是，我们最初的操作顺序是：

1. **日志写入：**把事务写入日志，包括一个事务开始块（transaction begin block）、所有待完成的数据和元数据更新，以及一个事务结束块（transaction end block）；等待这些写入完成。
2. **检查点：**把待完成的元数据和数据更新写到它们在文件系统中的最终位置。

在这个例子中，我们会先把 TxB、I[v2]、B[v2]、Db 和 TxE 写到日志里。这些写入完成后，再通过检查点（checkpoint）把 I[v2]、B[v2] 和 Db 写到它们在磁盘上的最终位置，从而完成这次更新。

如果在写日志的过程中发生了崩溃，事情就有点棘手了。这里我们要把事务中的一组块（例如 TxB、I[v2]、B[v2]、Db、TxE）写到磁盘上。一种简单的办法是一次发出一个写入，等它完成后再发出下一个。但这样太慢了。理想情况下，我们希望一次发出全部五个块的写入，这样就能把五次写入变成一次顺序写入，速度更快。然而，这样做并不安全，原因如下：面对这样一次大的写入，磁盘内部可能会做调度，按任意顺序完成这次大写入的各个小部分。因此，磁盘内部可能会（1）先写入 TxB、I[v2]、B[v2] 和 TxE，过一会儿才（2）写入 Db。遗憾的是，如果磁盘在（1）和（2）之间断电，磁盘上最终会是这样：



为什么这是个问题？嗯，这个事务看起来是一个有效的事务（它有开始块和结束块，而且序号相互匹配）。此外，文件系统没法看一眼第四个块就知道它是错的；毕竟，那是任意的用户数据。因此，如果系统现在重启并运行恢复，它就会重放（replay）这个事务，稀里糊涂地把垃圾块“??”的内容复制到 Db 本该存放的位置。对于文件里的任意用户数据，这

补充：强制写入磁盘

为了在两次磁盘写入之间强制规定顺序，现代文件系统必须采取一些额外的预防措施。在过去，要在两次写入 A 和 B 之间强制规定顺序很容易：只要向磁盘发出 A 的写入，等磁盘在写入完成时中断（interrupt）操作系统，然后再发出 B 的写入。

由于磁盘内部越来越多地使用写缓存（write cache），事情变得稍微复杂了一些。启用写缓冲（有时叫作 **immediate reporting**）后，磁盘只要把数据放进自己的内存缓存（cache），还没有真正写到磁盘上，就会告诉操作系统写入已经完成。如果操作系统接着发出后续的写入，就无法保证它会在前面的写入之后到达磁盘；因此，写入之间的顺序得不到保证。一种解决办法是禁用写缓冲（write buffering）。然而，更现代的系统会采取额外的预防措施，发出显式的**写屏障**（write barrier）；这样的屏障（barrier）完成时，能保证在屏障之前发出的所有写入，都会先于屏障之后发出的任何写入到达磁盘。

所有这些机制都要求我们非常信任磁盘能正确运行。遗憾的是，最近的研究表明，有些磁盘制造商为了交付“性能更高”的磁盘，明确地忽略写屏障（write barrier）请求，从而让磁盘看起来跑得更快，代价是有可能运行出错 [C+13, R+11]。正如 Kahan 所说，快的几乎总能打败慢的，哪怕快的是错的。

已经很糟糕了；如果这种事发生在文件系统的某个关键部分上，比如超级块，那就更糟了，文件系统可能因此无法挂载。

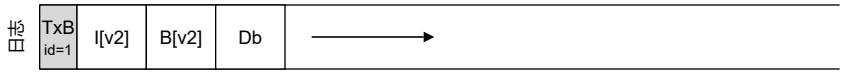
补充：优化日志写入

你可能已经注意到，写日志有一个特别低效的地方：文件系统必须先写出事务开始块（transaction begin block）和事务的内容；只有在这些写入完成之后，文件系统才能把事务结束块（transaction end block）发往磁盘。想想磁盘的工作原理，性能上的影响就很清楚了：通常会多花一圈旋转的时间（想想为什么）。

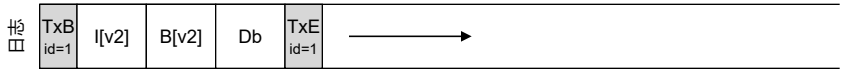
我们以前的一位研究生 Vijayan Prabhakaran 想出了一个简单的办法来解决这个问题 [P+05]。把事务写入日志时，在开始块（begin block）和结束块（end block）里加上日志内容的校验和（checksum）。这样一来，文件系统就可以一次写出整个事务，而不必等待；如果在恢复过程中，文件系统发现计算出的校验和（computed checksum）与事务中保存的校验和不一致，就可以断定崩溃发生在写入这个事务的过程中，从而丢弃这次文件系统更新。因此，只需对写入协议（protocol）和恢复系统稍作调整，文件系统就能在常见情况（common case）下获得更好的性能；除此之外，系统也更可靠了一点，因为现在从日志读出的任何内容都有校验和保护。

这个简单的修复很有吸引力，引起了 Linux 文件系统开发者的注意，他们随后把它纳入了下一代 Linux 文件系统，叫作（你猜对了！）**Linux ext4**。它现在运行在全世界数以百万计的机器上，包括 Android 手持平台。因此，在许多基于 Linux 的系统上，每当你向磁盘写入数据时，都有一小段在威斯康星开发的代码让你的系统变得更快、更可靠一点。

为了避免这个问题，文件系统分两步发出事务写入。首先，它把除 TxE 块之外的所有块写入日志，这些写入一次全部发出。这些写入完成后，日志看起来是这样的（还是假设我们的追加工作负载）：



等它们完成后，文件系统再发出 TxE 块的写入，使日志处于最终的安全状态：



这个过程的一个重要方面是磁盘提供的原子性（atomicity）保证。磁盘能保证任何 512 字节的写入要么完成，要么不发生（绝不会只写一半）；因此，为了确保 TxE 的写入是原子的，应该让它只占一个 512 字节的块。于是，我们目前更新文件系统的协议如下，三个阶段各有标注：

1. **日志写入：**把事务的内容（包括 TxB、元数据和数据）写入日志；等待这些写入完成。
2. **日志提交：**把事务提交块（transaction commit block，包含 TxE）写入日志；等待写入完成；此时称事务已**提交**（committed）。
3. **检查点：**把更新的内容（元数据和数据）写到它们在磁盘上的最终位置。

恢复

现在我们来理解文件系统如何利用日志的内容从崩溃中**恢复**（recover）。在这一系列更新中，崩溃随时可能发生。如果崩溃发生在事务安全地写入日志之前（即上面的第 2 步完成之前），我们的工作就很简单：直接跳过这次待完成的更新。如果崩溃发生在事务已经提交到日志之后、检查点（checkpoint）完成之前，文件系统可以按如下方式**恢复**这次更新。系统启动时，文件系统的恢复过程会扫描日志，找出已经提交到磁盘的事务；然后（按顺序）**重放**（replay）这些事务，文件系统再次尝试把事务中的块写到它们在磁盘上的最终位置。这种形式的日志是最简单的形式之一，叫作**重做日志**（redo logging）。恢复了日志中已提交的事务，文件系统就能确保磁盘上的结构是一致的，然后就可以继续挂载文件系统，准备处理新的请求了。

注意，在检查点（checkpoint）过程中的任何时刻发生崩溃都没有关系，即使有些块已经更新到了最终位置也无妨。最坏的情况是，其中一些更新在恢复时会再执行一遍。由于恢复是一种罕见的操作（只在系统意外崩溃之后才进行），多出几次冗余写入不值得担心[3]。

[3]除非你什么都担心，那我们也帮不了你。别这么操心了，这对健康不好！不过现在你大概又开始担心自己操心过度了。

批量更新日志

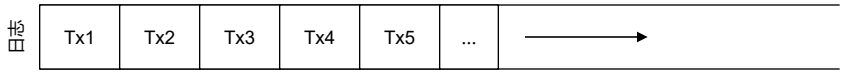
你可能已经注意到，这个基本协议会增加大量额外的磁盘流量。例如，假设我们在同一个目录里连续创建两个文件，分别叫作 `file1` 和 `file2`。创建一个文件，就要更新磁盘上的一些结构，至少包括：inode 位图（分配一个新 inode）、新创建的文件的 inode、父目录（parent directory）中包含新目录项的数据块，以及父目录的 inode（它现在有了新的修改时间）。有了日志，这两次文件创建在逻辑上都要把所有这些信息提交到日志；由于这两个文件在同一个目录中，再假设它们的 inode 甚至在同一个 inode 块（inode block）里，这就意味着，如果不加注意，我们就会一遍又一遍地写同样的这些块。

为了解决这个问题，有些文件系统（例如 Linux ext3）不会把每次更新一个一个地提交到磁盘，而是把所有更新缓冲到一个全局事务中。在上面的例子中，创建这两个文件时，文件系统只是把内存中的 inode 位图、文件的 inode、目录数据和目录 inode 标记为脏，并把它们加到构成当前事务的块列表中。等到最终该把这些块写到磁盘时，比如 5 秒超时（timeout）之后，就提交这个包含上述所有更新的全局事务。这样把更新缓冲起来，文件系统在许多情况下都能避免向磁盘写入过多的数据。

让日志大小有限

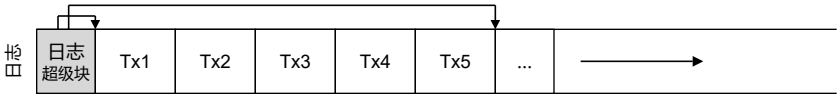
至此，我们得到了一个更新文件系统磁盘结构的基本协议。文件系统先把更新在内存中缓冲一段时间；等到最终该写入磁盘时，就小心地把事务的细节写到日志中，也就是预写日志（write-ahead logging）；事务完成后，再通过检查点（checkpoint）把这些块写到它们在磁盘上的最终位置。

然而，日志的大小是有限的。如果我们不停地往里面添加事务（就像下图这样），它很快就会被填满。你觉得那时会发生什么？



日志被填满时，会出现两个问题。第一个比较简单，但不那么要紧：日志越大，恢复所需的时间就越长，因为恢复时必须（按顺序）重放（replay）日志中的所有事务。第二个问题更严重：日志满了（或者快满了）时，就再也不能向磁盘提交新的事务了，文件系统也就变得“不太有用”（也就是没用了）。

为了解决这些问题，日志文件系统（journaling file system）把日志当作一个循环数据结构，反复使用它；这就是日志有时也叫**循环日志**（circular log）的原因。为此，文件系统必须在检查点（checkpoint）之后的某个时候采取行动。具体来说，一旦某个事务完成了检查点（checkpoint），文件系统就应该释放它在日志中占用的空间，让这些日志空间可以重新使用。实现这一目标的方法有很多；例如，你可以简单地在一个**日志超级块**（journal superblock）中标记出日志中最老的和最新的尚未完成检查点（checkpoint）的事务；其余空间都是空闲的。下面是图示：



在日志超级块（journal superblock）中（不要和文件系统主超级块混淆），日志系统记录足够的信息，以便知道哪些事务还没有完成检查点（checkpoint），从而既缩短了恢复时间，又能让日志循环使用。于是，我们在基本协议中又加了一步：

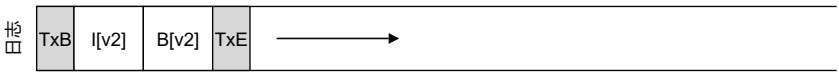
1. **日志写入：**把事务的内容（包含 TxB 和更新的内容）写入日志；等待这些写入完成。
2. **日志提交：**把事务提交块（transaction commit block，包含 TxE）写入日志；等待写入完成；事务现在已**提交**。
3. **检查点：**把更新的内容写到它们在文件系统中的最终位置。
4. **释放：**过一段时间后，更新日志超级块（journal superblock），把这个事务在日志中标记为空闲。

这样，我们就得到了最终的数据日志（data journaling）协议。但还有一个问题：我们把每个数据块都向磁盘写了两次，这是一笔沉重的代价，尤其是为了系统崩溃这样罕见的事情。你能想出一种不用把数据写两次、又能保持一致性的办法吗？

元数据日志

虽然恢复现在很快了（扫描日志并重放几个事务，而不是扫描整个磁盘），但文件系统的正常运行比我们希望的要慢。特别是，每次写入磁盘时，我们现在还要先写日志，写入流量因此翻了一倍；在顺序写的工作负载中，这种翻倍尤其让人头疼，因为写入速度现在只有驱动器峰值写入带宽（bandwidth）的一半。此外，在写日志和写主文件系统之间，还有一次代价高昂的寻道（seek），这在某些工作负载下会带来明显的开销。

由于把每个数据块都向磁盘写两次的代价很高，人们尝试了几种不同的办法来提升性能。例如，我们上面描述的日志模式通常叫作**数据日志**（data journaling，如 Linux ext3 中的叫法），因为它把所有用户数据（以及文件系统的元数据）都记入了日志。一种更简单（也更常见）的日志形式有时叫作**有序日志**（ordered journaling，或者就叫**元数据日志**，metadata journaling），它和数据日志（data journaling）几乎一样，只是用户数据不写入日志。因此，执行和上面一样的更新时，写入日志的是下面这些信息：



以前写入日志的数据块 Db，现在直接写到文件系统本身，避免了额外的写入；由于磁盘的大部分 I/O 流量都是数据，不把数据写两次就能大大减轻日志带来的 I/O 负担。不过，这一改动确实引出了一个有意思的问题：我们应该在什么时候把数据块写到磁盘上？

 © 2008–23, ARPACI-DUSSEAU

为了更好地理解这个问题，我们再来看看追加文件的那个例子。这次更新包括三个块：I[v2]、B[v2] 和 Db。前两个都是元数据，会先记入日志，再做检查点（checkpoint）；后一个只会向文件系统写入一次。我们应该什么时候把 Db 写到磁盘上？这有关系吗？

对于只记元数据的日志来说，数据写入的顺序确实很重要。例如，如果我们在事务（包含 I[v2] 和 B[v2]）完成之后才把 Db 写到磁盘，会怎样？遗憾的是，这种做法有一个问题：文件系统是一致的，但 I[v2] 最终可能指向垃圾数据。具体来说，考虑 I[v2] 和 B[v2] 写入了、但 Db 没能写到磁盘的情况。接着文件系统会尝试恢复。由于 Db 不在日志里，文件系统会重放（replay）对 I[v2] 和 B[v2] 的写入，得到一个（从文件系统元数据的角度看）一致的文件系统。然而，I[v2] 会指向垃圾数据，也就是 Db 本该写入的那个位置上原来的内容。

为了确保不出现这种情况，有些文件系统（例如 Linux ext3）会在相关的元数据写到磁盘之前，先把（普通文件的）数据块写到磁盘。具体来说，协议如下：

1. **数据写入：** 把数据写到最终位置；等待完成（这一等待是可选的，详见下文）。
2. **日志元数据写入：** 把开始块（begin block）和元数据写入日志；等待这些写入完成。
3. **日志提交：** 把事务提交块（transaction commit block，包含 TxE）写入日志；等待写入完成；事务现在已**提交**，包括数据在内。
4. **检查点元数据：** 把元数据更新的内容写到它们在文件系统中的最终位置。
5. **释放：** 之后，在日志超级块（journal superblock）中把这个事务标记为空闲。

强制先写数据，文件系统就能保证指针永远不会指向垃圾。其实，“先写被指向的对象，再写指向它的对象”这条规则正是崩溃一致性（crash consistency）的核心，其他一些崩溃一致性方案对它的利用还要更进一步 [GP94]（详见下文）。

在大多数系统中，元数据日志（类似于 ext3 中的有序日志）比完整的数据日志（data journaling）更流行。例如，Windows NTFS 和 SGI 的 XFS 都使用某种形式的元数据日志（metadata journaling）。Linux ext3 让你可以在数据（data）、有序（ordered）和无序（unordered）三种模式中任选一种（在无序模式下，数据可以在任何时候写入）。所有这些模式都能保持元数据一致；它们的区别在于对数据提供的语义不同。

最后要注意，正如上面的协议中所指出的，在发出日志写入（第 2 步）之前强制等待数据写入完成（第 1 步），并不是正确性所必需的。具体来说，同时发出对数据、事务开始块（transaction begin block）和日志元数据的写入也完全没问题；唯一真正的要求是，第 1 步和第 2 步要在发出日志提交块（第 3 步）之前完成。

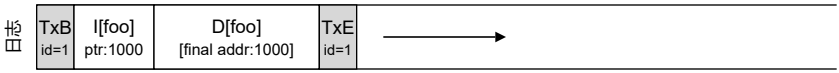
棘手的情况：块复用

还有一些有意思的边界情况会让日志变得更加棘手，因此值得讨论一下。其中有不少都和块复用有关；正如 Stephen Tweedie（ext3 背后的主要推动者之一）所说：

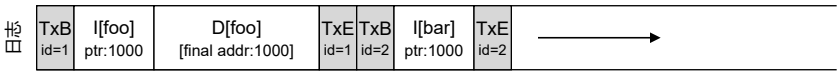
> “整个系统最可怕的部分是什么？……是删除文件。和删除有关的一切都很麻烦。和删除有关的一切……你会做噩梦，梦见块被删除、然后又被重新分配会发生什么。”[T00]

Tweedie 举的具体例子如下。假设你在使用某种形式的元数据日志（因此文件的数据块不记入日志）。比如说，有一个叫 `foo` 的目录。用户往 `foo` 里加了一个目录项（比如创建

了一个文件），于是 foo 的内容（因为目录被视为元数据）写进了日志；设 foo 目录数据存放在块 1000。日志里的内容大概是这样的：



这时，用户删除了这个目录里的所有内容以及目录本身，块 1000 被释放出来，可以重新使用。最后，用户创建了一个新文件（比如叫 bar），它恰好复用了原来属于 foo 的那个块（1000）。bar 的 inode 和它的数据都提交到了磁盘；但要注意，由于使用的是元数据日志（metadata journaling），只有 bar 的 inode 提交到了日志中；文件 bar 写入块 1000 的新数据没有记入日志。



现在假设发生了崩溃，而所有这些信息都还在日志里。重放（replay）时，恢复过程只是简单地重放日志中的所有内容，包括对块 1000 中目录数据的写入；于是，重放（replay）会用旧的目录内容覆盖当前文件 bar 的用户数据！显然，这不是正确的恢复操作，用户读文件 bar 时肯定会大吃一惊。

这个问题有几种解决办法。例如，可以规定：在这些块的删除操作通过检查点（checkpoint）移出日志之前，绝不复用这些块。Linux ext3 的做法则是在日志中加入一种新的记录类型，叫作 **revoke record**。在上面的例子中，删除目录时会往日志里写一条 revoke record。重放（replay）日志时，系统先扫描这类 revoke record；任何已被撤销（revoke）的数据都不会被重放（replay），从而避免了上面提到的问题。

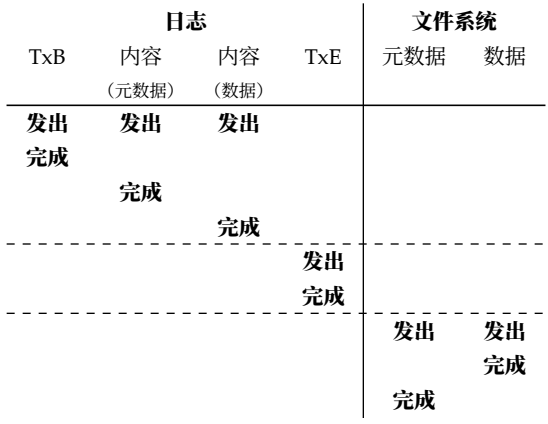


图 42.1：**数据日志的时间线**

 © 2008–23, ARPACI-DUSSEAU

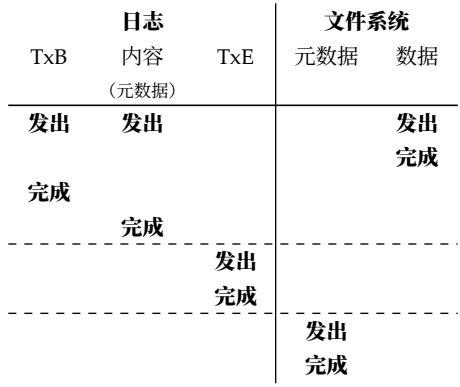


图 42.2：**元数据日志的时间线**

日志小结：时间线

在结束对日志的讨论之前，我们用时间线把讨论过的协议各总结一遍。图 42.1（第 13 页）展示的是对数据和元数据都记日志时的协议，图 42.2（第 14 页）展示的是只对元数据记日志时的协议。

在每张图中，时间向下递增，图中的每一行表示一次写入可以发出或可能完成的逻辑时间。例如，在数据日志（data journaling）协议中（图 42.1），事务开始块（TxB）的写入和事务内容的写入在逻辑上可以同时发出，因此可以按任意顺序完成；但是，对事务结束块（TxE）的写入，必须等前面这些写入完成之后才能发出。同样，对数据块和元数据块的检查点（checkpoint）写入，要等事务结束块（transaction end block）提交之后才能开始。水平虚线表示必须遵守写入顺序要求的地方。

元数据日志（metadata journaling）协议也有一张类似的时间线。注意，数据写入在逻辑上可以和事务开始块（transaction begin block）以及日志内容的写入同时发出；但是，它必须在事务结束块（transaction end block）发出之前发出并完成。

最后要注意，时间线中为每次写入标出的完成时间是任意的。在真实的系统中，完成时间由 I/O 子系统决定，它可能会为了提高性能而对写入重新排序。我们对顺序仅有的保证，就是为了协议的正确性而必须强制执行的那些（在图中用水平虚线表示）。

42.4 方案三：其他方法

到目前为止，我们介绍了保持文件系统元数据一致的两种方案：一种是基于 fsck 的“偷懒”方法，另一种是更主动的方法，叫作日志。然而，方法并不只有这两种。其中一种方法叫作软更新（Soft Updates）[GP94]，由 Ganger 和 Patt 提出。这种方法仔细安排所有写入文件系统的顺序，确保磁盘上的结构永远不会处于不一致的状态。例如，先把数据块写到磁盘，再写指向它的 inode，就能确保 inode 永远不会指向垃圾；对文件系统的所有结构，都可以推导出类似的规则。不过，实现软更新（soft updates）可能很有挑战性；上面描述的日

志层在实现时只需要对文件系统的具体结构有比较少的了解，而软更新（soft updates）则需要对每个文件系统数据结构都有细致入微的了解，因此会给系统增加相当多的复杂性。

另一种方法叫作**写时复制**（copy-on-write，没错，就是 **COW**），许多流行的文件系统都在使用它，包括 Sun 的 ZFS [B07]。这种技术从不原地覆盖文件或目录，而是把新的更新放到磁盘上以前没有使用过的位置。完成若干次更新之后，COW 文件系统会翻转文件系统的根结构，让它指向刚更新过的那些结构。这样一来，保持文件系统一致就很简单了。在后面的章节讨论日志结构文件系统（LFS）时，我们会更多地了解这种技术；LFS 就是 COW 的一个早期例子。

还有一种方法是我们在威斯康星这里刚刚开发出来的。这种技术叫作 **backpointer-based consistency**（**BBC**），它不在写入之间强制规定任何顺序。为了实现一致性，系统中的每个块都会额外加上一个**反向指针**（back pointer）；例如，每个数据块都引用它所属的 inode。访问文件时，文件系统可以检查正向指针（例如 inode 或直接块中的地址）所指向的块是否反过来引用了它，以此判断这个文件是否一致。如果是，那么一切一定都已安全地写到了磁盘上，文件是一致的；如果不是，文件就不一致，文件系统返回一个错误。给文件系统加上反向指针（back pointer），就能实现一种新形式的“偷懒”式崩溃一致性（crash consistency） [C+12]。

最后，我们还研究了一些技术，用来减少日志协议必须等待磁盘写入完成的次数。这种新方法叫作 **optimistic crash consistency** [C+13]，它利用一种推广形式的**事务校验和**（transaction checksum）[P+05]，尽可能多地向磁盘发出写入，并用另外几种技术来检测可能出现的不一致。对于某些工作负载，这些乐观的技术能把性能提高一个数量级。然而，要真正发挥作用，需要一个稍有不同的磁盘接口 [C+13]。

42.5 小结

我们介绍了崩溃一致性问题（crash-consistency problem），并讨论了解决这个问题的各种方法。较老的方法是构建一个文件系统检查程序（file system checker），它能用，但在现代系统上恢复起来可能太慢了。因此，现在许多文件系统改用了日志。日志把恢复时间从 O(磁盘卷的大小) 减少到 O(日志的大小)，从而大大加快了崩溃重启后的恢复速度，这也是许多现代文件系统采用日志的原因。我们还看到，日志可以有许多不同的形式；最常用的是有序元数据日志（metadata journaling），它减少了写入日志的流量，同时仍然对文件系统元数据和用户数据都提供了合理的一致性保证。归根结底，对用户数据提供强有力的保证，可能是最重要的事情之一；说来也怪，正如最近的研究所表明的，这个领域仍然在不断发展 [P+14]。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[B07] “ZFS: The Last Word in File Systems” by Jeff Bonwick and Bill Moore. Available online: http://www.ostep.org/Citations/zfs_last.pdf. *ZFS uses copy-on-write and journaling, actually, as in some cases, logging writes to disk will perform better.*

[C+12] “Consistency Without Ordering” by Vijay Chidambaram, Tushar Sharma, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST ’12, San Jose, California. *A recent paper of ours about a new form of crash consistency based on back pointers. Read it for the exciting details!*

[C+13] “Optimistic Crash Consistency” by Vijay Chidambaram, Thanu S. Pillai, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau . SOSP ’13, Nemacolin Woodlands Resort, PA, November 2013. *Our work on a more optimistic and higher performance journaling protocol. For workloads that call* `fsync()` *a lot, performance can be greatly improved.*

[GP94] “Metadata Update Performance in File Systems” by Gregory R. Ganger and Yale N. Patt. OSDI ’94. *A clever paper about using careful ordering of writes as the main way to achieve consistency. Implemented later in BSD-based systems.*

[G+08] “SQCK: A Declarative File System Checker” by Haryadi S. Gunawi, Abhishek Rajimwale, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI ’08, San Diego, California. *Our own paper on a new and better way to build a file system checker using SQL queries. We also show some problems with the existing checker, finding numerous bugs and odd behaviors, a direct result of the complexity of* `fsck`.

[H87] “Reimplementing the Cedar File System Using Logging and Group Commit” by Robert Hagmann. SOSP ’87, Austin, Texas, November 1987. *The first work (that we know of) that applied write-ahead logging (a.k.a. journaling) to a file system.*

[M+13] “ffsck: The Fast File System Checker” by Ao Ma, Chris Dragga, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. FAST ’13, San Jose, California, February 2013. *A recent paper of ours detailing how to make fsck an order of magnitude faster. Some of the ideas have already been incorporated into the BSD file system checker [MK96] and are deployed today.*

[MK96] “Fsck – The UNIX File System Check Program” by Marshall Kirk McKusick and T. J. Kowalski. Revised in 1996. *Describes the first comprehensive file-system checking tool, the eponymous* `fsck`. *Written by some of the same people who brought you FFS.*

[MJLF84] “A Fast File System for UNIX” by Marshall K. McKusick, William N. Joy, Sam J. Leffler, Robert S. Fabry. ACM Transactions on Computing Systems, Volume 2:3, August 1984. *You already know enough about FFS, right? But come on, it is OK to re-reference important papers.*

[P+14] “All File Systems Are Not Created Equal: On the Complexity of Crafting Crash-Consistent Applications” by Thanumalayan Sankaranarayana Pillai, Vijay Chidambaram, Ramnatthan Alagappan, Samer Al-Kiswany, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. OSDI ’14, Broomfield, Colorado, October 2014. *A paper in which we study what file systems guarantee after crashes, and show that applications expect something different, leading to all sorts of interesting problems.*

[P+05] “IRON File Systems” by Vijayan Prabhakaran, Lakshmi N. Bairavasundaram, Nitin Agrawal, Haryadi S. Gunawi, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. SOSP ’05, Brighton, England, October 2005. *A paper mostly focused on studying how file systems react to disk failures. Towards the end, we introduce a transaction checksum to speed up logging, which was eventually adopted into Linux ext4.*

[PAA05] “Analysis and Evolution of Journaling File Systems” by Vijayan Prabhakaran, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. USENIX ’05, Anaheim, California, April 2005. *An early paper we wrote analyzing how journaling file systems work.*

[R+11] “Coerced Cache Eviction and Discreet-Mode Journaling” by Abhishek Rajimwale, Vijay Chidambaram, Deepak Ramamurthi, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. DSN ’11, Hong Kong, China, June 2011. *Our own paper on the problem of disks that buffer writes in a memory cache instead of forcing them to disk, even when explicitly told not to do that! Our solution to overcome this problem: if you want A to be written to disk before B, first write A, then send a lot of “dummy” writes to disk, hopefully causing A to be forced to disk to make room for them in the cache. A neat if impractical solution.*

[T98] “Journaling the Linux ext2fs File System” by Stephen C. Tweedie. The Fourth Annual Linux Expo, May 1998. *Tweedie did much of the heavy lifting in adding journaling to the Linux ext2 file system; the result, not surprisingly, is called ext3. Some nice design decisions include the strong focus on backwards compatibility, e.g., you can just add a journaling file to an existing ext2 file system and then mount it as an ext3 file system.*

 © 2008–23, ARPACI-DUSSEAU

[T00] “EXT3, Journaling Filesystem” by Stephen Tweedie. Talk at the Ottawa Linux Symposium, July 2000. olstrans.sourceforge.net/release/OLS2000-ext3/OLS2000-ext3.html *A transcript of a talk given by Tweedie on ext3.*

[T01] “The Linux ext2 File System” by Theodore Ts’o, June, 2001.. Available online here: http://e2fsprogs.sourceforge.net/ext2.html. *A simple Linux file system based on the ideas found in FFS. For a while it was quite heavily used; now it is really just in the kernel as an example of a simple file system.*

 © 2008–23, ARPACI-DUSSEAU

作业（模拟）

本节介绍 `fsck.py`，这是一个简单的模拟器，可以帮你更好地理解如何检测（并可能修复）文件系统损坏。关于如何运行这个模拟器，请参阅附带的 README。

问题

1. 首先，运行 `fsck.py -D`；这个选项会关闭所有损坏，因此你可以用它生成一个随机的文件系统，看看你能否判断出里面有哪些文件和目录。那就动手吧！用 `-p` 选项看看你是否答对了。把随机种子（`-s`）设成不同的值，比如 1、2 和 3，对几个不同的随机生成的文件系统试一试。
2. 现在，我们来引入一处损坏。先运行 `fsck.py -S 1`。你能看出引入了什么不一致吗？在真实的文件系统修复工具中，你会怎样修复它？用 `-c` 检查你是否答对了。
3. 把种子改成 `-S 3` 或 `-S 19`；你看到了什么不一致？用 `-c` 检查你的答案。这两种情况有什么不同？
4. 把种子改成 `-S 5`；你看到了什么不一致？自动修复这个问题会有多难？用 `-c` 检查你的答案。然后，用 `-S 38` 引入一个类似的不一致；这个更难检测吗？能检测出来吗？最后，用 `-S 642`；这个不一致能检测出来吗？如果能，你会怎样修复这个文件系统？
5. 把种子改成 `-S 6` 或 `-S 13`；你看到了什么不一致？用 `-c` 检查你的答案。这两种情况有什么区别？修复工具遇到这种情况时应该怎么做？
6. 把种子改成 `-S 9`；你看到了什么不一致？用 `-c` 检查你的答案。在这种情况下，检查修复工具应该相信哪一部分信息？
7. 把种子改成 `-S 15`；你看到了什么不一致？用 `-c` 检查你的答案。在这种情况下，修复工具能做什么？如果无法修复，会丢失多少数据？
8. 把种子改成 `-S 10`；你看到了什么不一致？用 `-c` 检查你的答案。这里的文件系统结构中有没有冗余（redundancy）信息可以帮助修复？
9. 把种子改成 `-S 16` 和 `-S 20`；你看到了什么不一致？用 `-c` 检查你的答案。修复工具应该怎样修复这个问题？

 © 2008–23, ARPACI-DUSSEAU